



Instituto Tecnológico de Aeronáutica - ITA
CM-204 - Aprendizado Profundo
Aluno: Ulisses Lopes da Silva

Relatório do Laboratório 05 - Generative Adversarial Networks (GANs)

1 Breve explicação em alto nível do que foi implementado

1.1 Gerador

O gerador é a rede responsável por criar imagens falsas a partir de um vetor de ruído aleatório (o *espaço latente*). Sua arquitetura funciona como um processo de *ampliação progressiva*: partindo de um vetor de 100 números, a rede aplica cinco camadas de convolução transposta — operação inversa à convolução, que aumenta as dimensões espaciais da imagem a cada etapa — até produzir uma imagem colorida de 64×64 pixels.

A primeira camada parte de uma representação 1×1 e a expande para 4×4 ; as camadas seguintes dobram as dimensões a cada passo (8×8 , 16×16 , 32×32 , 64×64), enquanto o número de canais é reduzido pela metade em cada etapa (512, 256, 128, 64, 3). Após cada camada intermediária, aplica-se normalização de lote (*Batch Normalization*) e a função de ativação ReLU. A camada final utiliza a função Tanh, que garante que os valores dos pixels fiquem no intervalo $[-1, 1]$.

```
class Generator(nn.Module):
    def __init__(self):
        super(Generator, self).__init__()
        self.network = nn.Sequential(
            # in: latent_size x 1 x 1
            nn.ConvTranspose2d(latent_size, 8 * size_multiplier, kernel_size
                               =4, stride=1, padding=0, bias=False),
            nn.BatchNorm2d(8 * size_multiplier),
            nn.ReLU(True),
            # tensor: 512 x 4 x 4
            nn.ConvTranspose2d(8 * size_multiplier, 4 * size_multiplier,
                               kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(4 * size_multiplier),
            nn.ReLU(True),
            # tensor: 256 x 8 x 8
            nn.ConvTranspose2d(4 * size_multiplier, 2 * size_multiplier,
                               kernel_size=4, stride=2, padding=1, bias=False),
            nn.BatchNorm2d(2 * size_multiplier),
            nn.ReLU(True),
            # tensor: 128 x 16 x 16
```

```

        nn.ConvTranspose2d(2 * size_multiplier, size_multiplier,
                           kernel_size=4, stride=2, padding=1, bias=False),
        nn.BatchNorm2d(size_multiplier),
        nn.ReLU(True),
        # tensor: 64 x 32 x 32
        nn.ConvTranspose2d(size_multiplier, num_color_channels,
                           kernel_size=4, stride=2, padding=1, bias=False),
        nn.Tanh(),
        # out: 3 x 64 x 64
    )

def forward(self, input: torch.Tensor) -> torch.Tensor:
    return self.network(input)

```

Listing 1: Implementação do Gerador

1.2 Discriminador

O discriminador é a rede que avalia se uma imagem é real (do conjunto de dados) ou falsa (gerada pelo gerador). Sua arquitetura é o espelho do gerador: cinco camadas convolucionais reduzem progressivamente as dimensões espaciais da imagem de entrada (64×64) até um único valor escalar, que representa a probabilidade de a imagem ser real.

O número de canais cresce a cada camada (64, 128, 256, 512) e a resolução espacial é reduzida pela metade a cada etapa. As camadas intermediárias utilizam a função LeakyReLU (com inclinação negativa de 0,2) e normalização de lote. A última camada produz um único número, transformado pela função Sigmoid em uma probabilidade entre 0 e 1.

```

class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.network = nn.Sequential(
            # in: 3 x 64 x 64
            nn.Conv2d(num_color_channels, size_multiplier, kernel_size=4,
                      stride=2, padding=1, bias=False),
            nn.BatchNorm2d(size_multiplier),
            nn.LeakyReLU(0.2, inplace=True),
            # tensor: 64 x 32 x 32
            nn.Conv2d(size_multiplier, 2 * size_multiplier, kernel_size=4,
                      stride=2, padding=1, bias=False),
            nn.BatchNorm2d(2 * size_multiplier),
            nn.LeakyReLU(0.2, inplace=True),
            # tensor: 128 x 16 x 16
            nn.Conv2d(2 * size_multiplier, 4 * size_multiplier, kernel_size=4,
                      stride=2, padding=1, bias=False),
            nn.BatchNorm2d(4 * size_multiplier),
            nn.LeakyReLU(0.2, inplace=True),
            # tensor: 256 x 8 x 8
            nn.Conv2d(4 * size_multiplier, 8 * size_multiplier, kernel_size=4,
                      stride=2, padding=1, bias=False),
            nn.BatchNorm2d(8 * size_multiplier),
            nn.LeakyReLU(0.2, inplace=True),

```

```

        # tensor: 512 x 4 x 4
        nn.Conv2d(8 * size_multiplier, 1, kernel_size=4, stride=1, padding
            =0, bias=False),
        # out: 1 x 1 x 1
        nn.Flatten(),
        nn.Sigmoid(),
    )

def forward(self, input: torch.Tensor) -> torch.Tensor:
    return self.network(input)

```

Listing 2: Implementação do Discriminador

1.3 Inicialização dos pesos

Seguindo a convenção da DCGAN, os pesos das camadas convolucionais e de convolução transposta são inicializados com uma distribuição normal de média zero e desvio padrão de 0,02. Os pesos das camadas de normalização de lote são inicializados com média 1,0 e mesmo desvio padrão, enquanto seus *biases* são inicializados em zero. Como as camadas convolucionais foram criadas sem *bias* (`bias=False`), não há inicialização de *bias* para elas.

```

def weights_init(m):
    classname = m.__class__.__name__
    if classname.find('Conv') != -1:
        nn.init.normal_(m.weight.data, 0.0, 0.02)
    elif classname.find('BatchNorm') != -1:
        nn.init.normal_(m.weight.data, 1.0, 0.02)
        nn.init.constant_(m.bias.data, 0)

```

Listing 3: Inicialização dos pesos

1.4 Treinamento do Discriminador

A cada iteração, o discriminador é treinado para distinguir imagens reais de falsas. Isso é feito em dois passos: primeiro, calcula-se a perda sobre um lote de imagens reais (cujo alvo é 1, ou seja, “real”); em seguida, o gerador produz imagens falsas a partir de vetores de ruído aleatório, e calcula-se a perda sobre essas imagens (cujo alvo é 0, ou seja, “falsa”). A perda total é a soma das duas, e o gradiente é propagado apenas pelo discriminador, com o gerador sendo mantido fixo nessa etapa.

```

def training_step_discriminator(discriminator, generator, real_images, opt_d):
    opt_d.zero_grad()

    real_preds = discriminator(real_images)
    real_targets = torch.ones(real_images.size(0), 1, device=device)
    real_loss = F.binary_cross_entropy(real_preds, real_targets)
    real_score = torch.mean(real_preds).item()

    latent = torch.randn(real_images.size(0), latent_size, 1, 1, device=
        device)

```

```

fake_images = generator(latent)
fake_preds = discriminator(fake_images.detach())
fake_targets = torch.zeros(real_images.size(0), 1, device=device)
fake_loss = F.binary_cross_entropy(fake_preds, fake_targets)
fake_score = torch.mean(fake_preds).item()

loss = real_loss + fake_loss
loss.backward()
opt_d.step()

return loss.item(), real_score, fake_score

```

Listing 4: Iteração de treinamento do discriminador

1.5 Treinamento do Gerador

O gerador é treinado para enganar o discriminador, ou seja, para que as imagens que ele produz sejam classificadas como reais. Para isso, gera-se um lote de imagens falsas e calcula-se a perda de entropia cruzada binária usando o alvo 1 (como se fossem reais). Diferentemente do treino do discriminador, o gradiente flui por toda a rede (inclusive pelo gerador) para que seus pesos sejam atualizados.

```

def training_step_generator(discriminator, generator, num_images, opt_g):
    opt_g.zero_grad()

    latent = torch.randn(num_images, latent_size, 1, 1, device=device)
    fake_images = generator(latent)
    fake_preds = discriminator(fake_images)
    fake_targets = torch.ones(num_images, 1, device=device)
    loss = F.binary_cross_entropy(fake_preds, fake_targets)

    loss.backward()
    opt_g.step()

    return loss.item()

```

Listing 5: Iteração de treinamento do gerador

1.6 Iteração do Treinamento Geral

A função `fit_step` combina as duas etapas anteriores em uma única iteração: primeiro treina o discriminador com o lote de imagens reais, e em seguida treina o gerador com o mesmo tamanho de lote. Os quatro valores retornados — perda do discriminador, perda do gerador, *score* das imagens reais e *score* das imagens falsas — são registrados ao longo do treinamento para acompanhar a evolução do modelo.

```

def fit_step(discriminator, generator, real_images, opt_d, opt_g):
    loss_d, real_score, fake_score = training_step_discriminator(
        discriminator, generator, real_images.to(device), opt_d
    )

```

```
loss_g = training_step_generator(  
    discriminator, generator, real_images.size(0), opt_g  
)  
return loss_d, loss_g, real_score, fake_score
```

Listing 6: Iteração geral do treinamento

2 Figuras Comprovando Funcionamento do Código

O treinamento foi feito com a GPU do Google Colab. Dessa forma, foi possível completar todas as 50 épocas e visualizar melhor os resultados. Abaixo seguem as imagens geradas em diferentes épocas do treinamento.

2.1 1ª Época

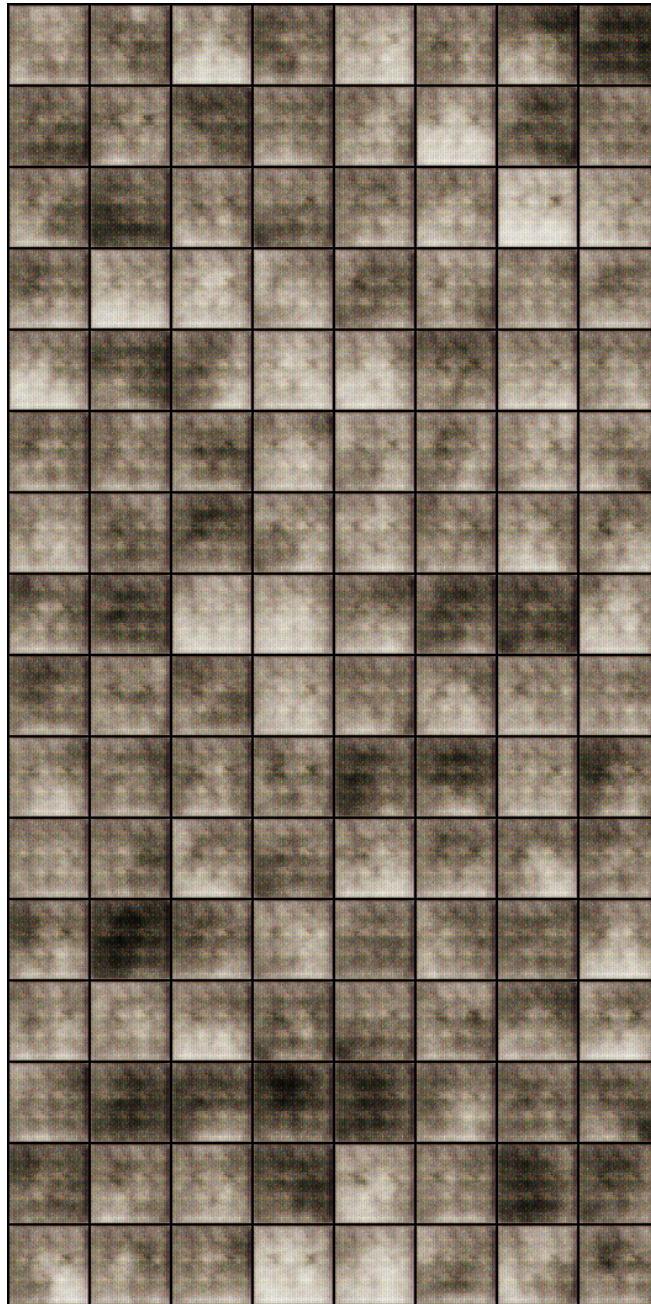


Fig. 1: Imagens geradas após a 1ª época. O gerador ainda não aprendeu nenhuma estrutura relevante; as imagens são compostas por ruído visual sem forma definida, com tonalidades acinzentadas uniformes.

2.2 2ª Época

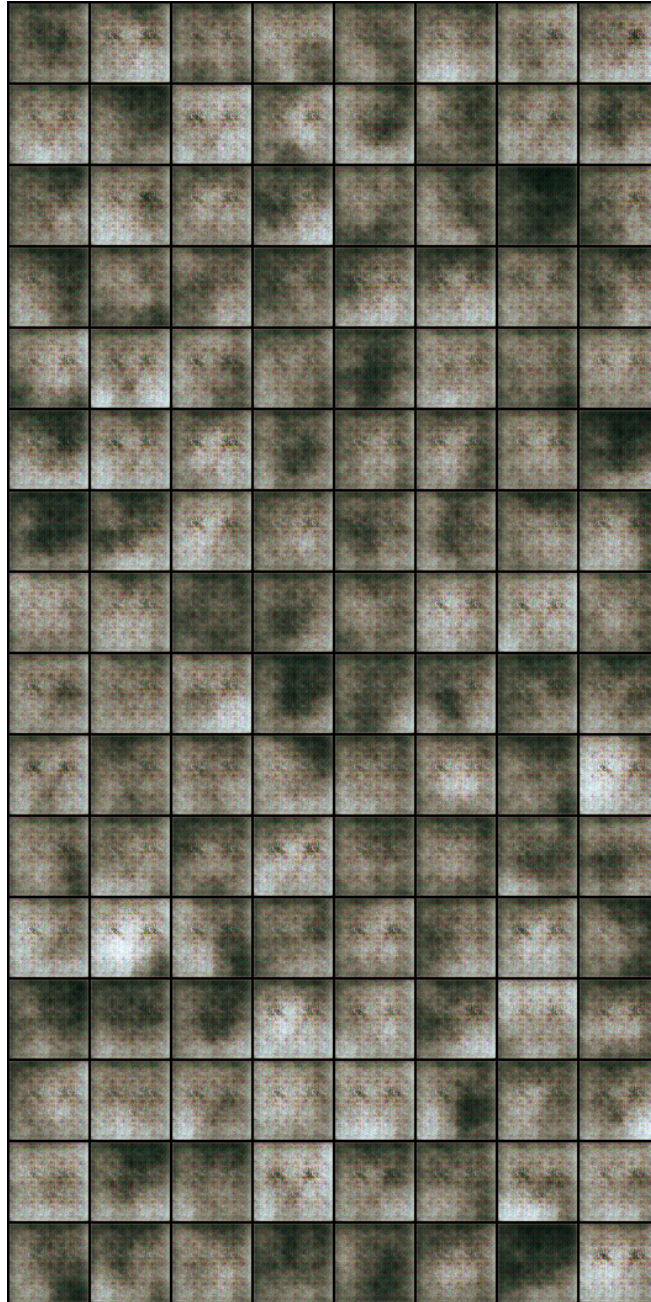


Fig. 2: Imagens geradas após a 2ª época. As imagens ainda apresentam ruído dominante, mas já é possível notar uma leve variação de textura e contraste em relação à época anterior, indicando que o gerador começa a captar padrões rudimentares de cor e iluminação.

2.3 10ª Época

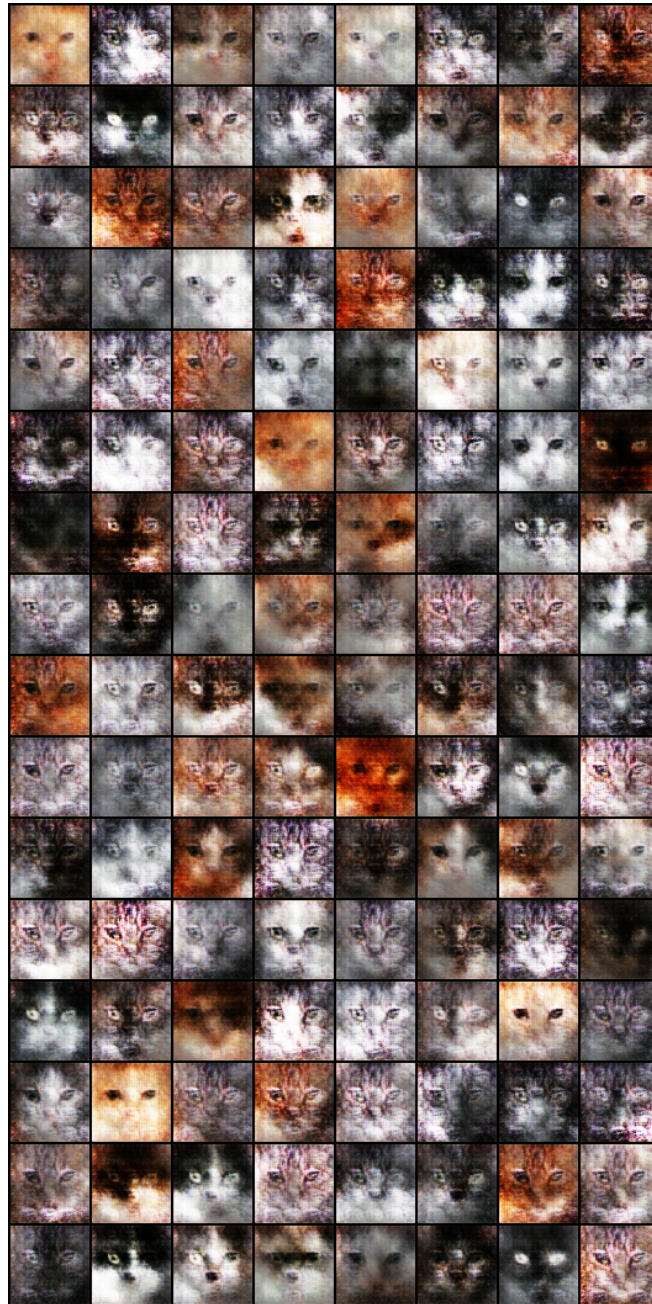


Fig. 3: Imagens geradas após a 10ª época. Neste ponto, o gerador já produz estruturas reconhecíveis: é possível identificar formas que lembram rostos de gatos, com olhos, nariz e pelagem em posições aproximadas. A qualidade ainda é baixa, mas o progresso é evidente.

2.4 50ª Época



Fig. 4: Imagens geradas após a 50ª época. As imagens apresentam rostos de gatos com detalhes mais definidos — olhos, pelo e proporções reconhecíveis — embora ainda com artefatos e deformações.

2.5 Perdas e Scores

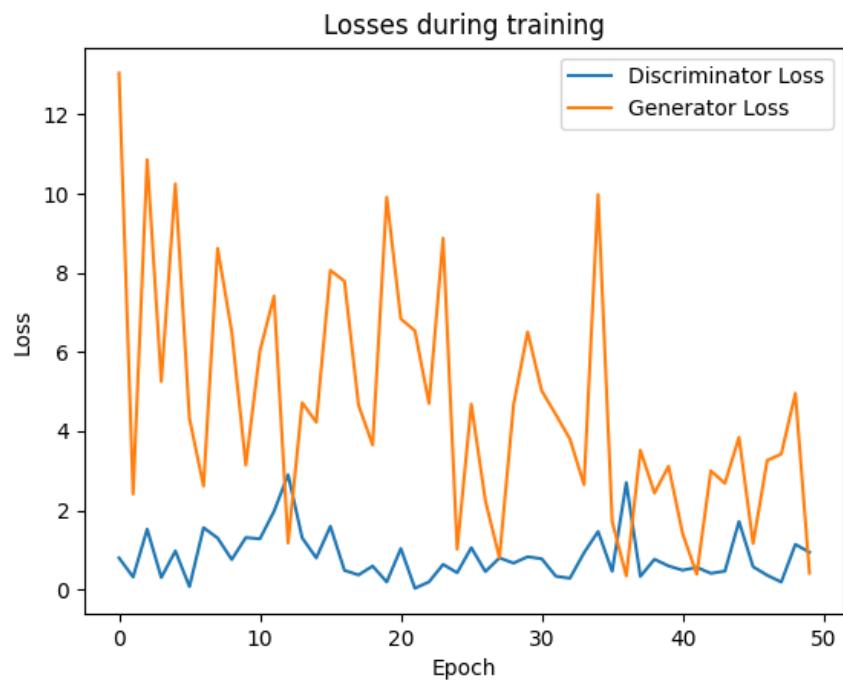


Fig. 5: Evolução das perdas do discriminador e do gerador ao longo das 50 épocas de treinamento.

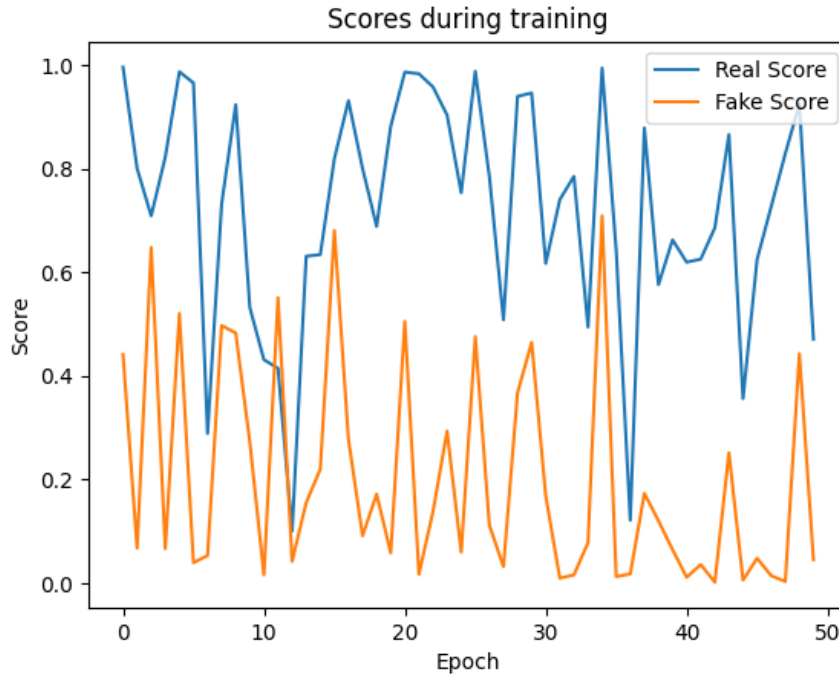


Fig. 6: Evolução dos *scores* atribuídos pelo discriminador às imagens reais e falsas ao longo das 50 épocas.

3 Discussão dos Resultados

- **Evolução visual das imagens geradas:** A progressão das Figs. 1 a 4 ilustra claramente o aprendizado do gerador. Na 1ª época, as saídas são puramente ruidosas e sem estrutura, talvez com uma leve nuance de formas de gatos, mas ainda muito inconsistentes. Já na 10ª época, rostos de gatos são reconhecíveis, e na 50ª época, os detalhes faciais — olhos, nariz, textura do pelo — estão bem mais definidos. A variedade de cores e padrões entre as amostras da 50ª época sugere que o gerador aprendeu uma distribuição diversificada, sem colapso de modo (*mode collapse*), problema comum em GANs no qual o gerador passa a produzir sempre imagens muito parecidas.
- **Perdas durante o treinamento:** A Fig. 5 mostra que a perda do discriminador se mantém relativamente baixa e estável ao longo do treinamento, enquanto a perda do gerador começa alta e apresenta oscilações consideráveis, mas com tendência de queda ao longo das épocas. Esse comportamento é esperado em GANs, em que o treinamento é inerentemente instável porque os dois modelos competem entre si, e picos na perda do gerador indicam momentos em que o discriminador está particularmente eficaz em rejeitar as imagens falsas.
- **Scores durante o treinamento:** A Fig. 6 mostra que o *score* das imagens reais permanece consistentemente alto (próximo de 1,0), enquanto o *score* das imagens falsas flutua em torno de valores mais baixos (0,1 a 0,5). Isso indica que o discriminador manteve uma vantagem sobre o gerador durante praticamente todo o treinamento, sendo capaz de

distinguir imagens reais das falsas com razoável confiança. Para um equilíbrio ideal, ambos os *scores* deveriam convergir para aproximadamente 0,5; a distância observada sugere que mais épocas de treinamento ainda trariam melhorias significativas na qualidade das imagens geradas.

- **Limitações:** Mesmo após 50 épocas, as imagens geradas ainda apresentam distorções visíveis. Isso é esperado para o número de épocas em que foi treinada a arquitetura. É provável que treinamentos mais longos produziram resultados substancialmente melhores. Ainda assim, o resultado obtido demonstra que a implementação está correta e que a rede aprendeu a capturar os traços essenciais da distribuição de imagens de gatos.